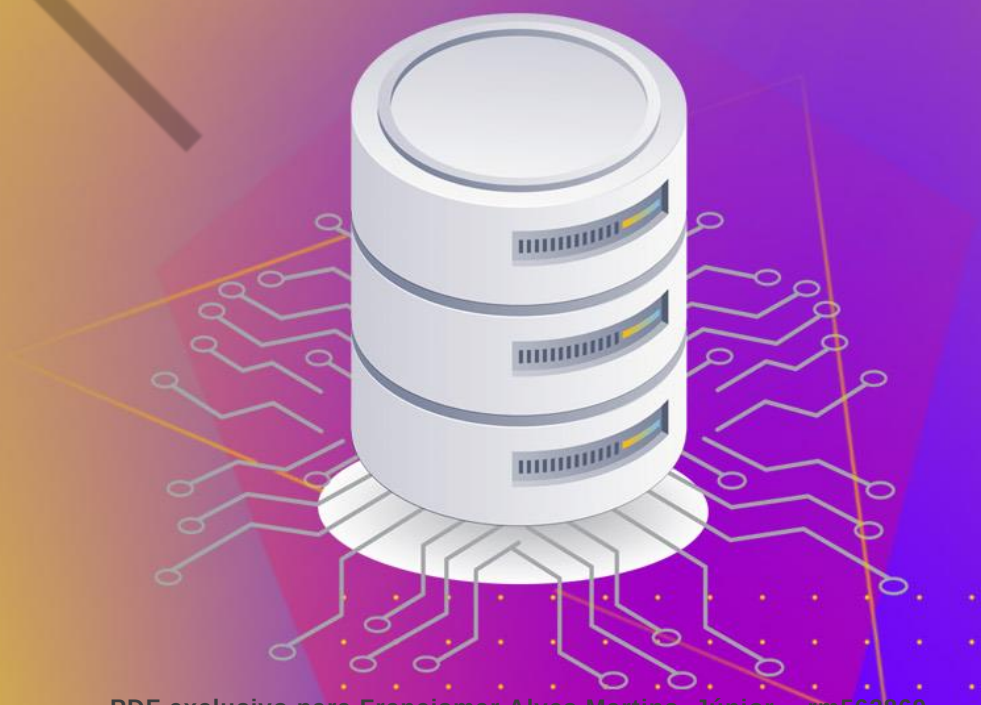
FIAP ON

COLHEITA DE DADOS E INSIGHTS - DADOS VALIOSOS E MADUROS

ARRUMAÇÃO INTERNA DOS DADOS



4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - DataFrame armazenado em arquivo CSV com separador utilizando ponto e vírgula 15
Figura 2 - Gráfico gerado diretamente a partir do DataFrame Pandas 20

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE TABELAS

Tabela 1 - DataFrame com ordenação ascendente pelo Produto 12
Tabela 2 - DataFrame com ordenação descendente pelo Produto 12
Tabela 3 - DataFrame com valores nulos excluídos 17
Tabela 4 - DataFrame com valores nulos preenchidos com a média da coluna 17
Tabela 5 - Resultado obtido com execução do método *describe* 18
Tabela 6 - Top 10 registros com média móvel de três dias 19

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de declaração da biblioteca 8
Código-fonte 2 - Exemplo de declaração de uma Series a partir de uma lista.....9
Código-fonte 3 - Exemplo de declaração de um DataFrame a partir de um dicionário 10
Código-fonte 4 - Acesso às Series e valores do DataFrame.....10
Código-fonte 5 - Filtragem de dados 11
Código-fonte 6 - Agregação de dados.....12
Código-fonte 7 - Ordenação de dados 12
Código-fonte 8 - Junção de DataFrames 13
Código-fonte 9 - Mesclagem de DataFrames.....14
Código-fonte 10 - Concatenação de DataFrames 14
Código-fonte 11 - Escrita e leitura de um arquivo CSV 15
Código-fonte 12 - Preparação e limpeza de dados 17
Código-fonte 13 - Análise estatística descritiva.....18
Código-fonte 14 - Análise de séries temporais com média móvel.....19
Código-fonte 15 - Script para criação de gráfico diretamente a partir do DataFrame 20

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Comando para instalação da biblioteca.....8

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 ARRUMAÇÃO INTERNA DOS DADOS 7
1.1 Introdução 7
1.2 Instalação 8
1.3 Como utilizar 8
2 PRINCIPAIS ESTRUTURAS DE DADOS 9
2.1 Series 9
2.2 DataFrame 9
3 OPERAÇÕES BÁSICAS 11
3.1 Filtragem de dados 11
3.2 Agregação de dados 11
3.3 Ordenação dos dados 12
4 COMBINAÇÃO DE DATAFRAMES 13
4.1 Junção (*Join*) 13
4.2 Mesclagem (*Merge*) 14
4.3 Concatenação (*Concat*) 14
5 LEITURA E ESCRITA DE ARQUIVOS 15
6 APLICAÇÕES PRÁTICAS 16
6.1 Limpeza e preparação dos dados 16
6.2 Análise de dados 17
6.3 Análise de séries temporais 18
6.4 Integração nativa com visualização de dados 20
CONCLUSÃO 21
REFERÊNCIAS 22

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 ARRUMAÇÃO INTERNA DOS DADOS

1.1 Introdução

Pandas é uma biblioteca de software escrita para a linguagem de programação Python, amplamente utilizada para manipulação e análise de dados. Criada em 2008, pelo desenvolvedor e empresário Wes McKinney, Pandas oferece estruturas de dados flexíveis e robustas, permitindo a manipulação eficiente de dados unidimensionais e bidimensionais em larga escala. A biblioteca possui uma ampla gama de funções para realizar operações comuns de análise de dados, como filtragem, agregação, combinação e agrupamento, além de possuir funcionalidades para manipulação de informações ausentes e limpeza de dados. Sua eficiência em operações vetorizadas e a capacidade de manipular grandes conjuntos de dados em memória tornam-na essencial para aplicações em um contexto de Big Data. Além disso, o Pandas também facilita a leitura e escrita de dados em diversos formatos, como CSV, Excel, SQL e JSON, integrando-se facilmente com outras ferramentas e sistemas.

Em termos de aplicações, Pandas é amplamente utilizado em diversas disciplinas e setores do mercado. Em Ciência de Dados, ele é essencial para o pré-processamento das amostras facilitando a limpeza e transformação de dados brutos em vetores de características, adequados para a interpretação dos modelos de Machine Learning. Na área da saúde, essa biblioteca é utilizada para analisar prontuários de pacientes, realizar estudos de epidemiologia e monitorar a eficácia de tratamentos. No setor financeiro, é aplicada na análise de séries temporais e modelagem de dados financeiros, facilitando o acompanhamento da cotação de ações, previsão de tendências e gerenciamento de portfólios. Na área de marketing e negócios, o Pandas é usado para auxiliar na análise de dados de vendas, segmentar clientes e avaliar o impacto de campanhas publicitárias. Sua flexibilidade e integração com outras bibliotecas robustas, como Matplotlib e Seaborn, tornam-na uma ferramenta indispensável para qualquer profissional envolvido em análise e visualização de dados, oferecendo um arcabouço completo para extração de *insights* valiosos, visando auxiliar a tomada de decisões estratégicas pautadas em dados (Pandas, 2024).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.2 Instalação

A biblioteca Pandas já vem instalada por padrão com a distribuição Anaconda. Porém, você também pode utilizar o gerenciador de pacotes pip para realizar uma instalação independente via terminal com o seguinte comando:

```
pip install pandas
```

Comando de prompt 1 - Comando para instalação da biblioteca
Fonte: Elaborado pelo autor (2024)

1.3 Como utilizar

Para começar a utilizar o Pandas, é necessário importar a biblioteca em seu script Python. A convenção é importá-la seguida pelo codinome “pd”, facilitando o acesso posterior às suas funções e métodos, como especificado a seguir:

```
import pandas as pd
```

Código-fonte 1 - Exemplo de declaração da biblioteca
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 PRINCIPAIS ESTRUTURAS DE DADOS

O Pandas fornece duas principais estruturas de dados para a manipulação e análise de dados: **Series** e **DataFrame**. A Series pode ser considerada como uma coluna em uma tabela, onde cada elemento é associado a um índice. O DataFrame, por outro lado, pode ser comparado a uma tabela de banco de dados, com linhas e colunas rotuladas. Ambas são projetadas para facilitar a análise e manipulação de dados de forma eficiente, suportando operações complexas com uma sintaxe intuitiva e flexível, o que as tornam fundamentais para a ciência de dados e análise estatística.

2.1 Series

As Series são estruturas de dados unidimensionais que podem armazenar qualquer tipo de dados, sejam eles inteiros, *strings*, *floats*, objetos, entre outros. Elas são semelhantes aos *arrays* do NumPy, mas com a vantagem de possuírem rótulos para cada elemento. O acesso aos dados de uma Series é feito por meio de um índice numérico equivalente às listas nativas do Python e aos *arrays* da biblioteca NumPy.

```
import pandas as pd

# Criando uma Series a partir de uma lista
s = pd.Series([1, 2, 3, 4, 5])

# Acesso à posição de índice 4
print(s[4]) # Saída: 5
```

Código-fonte 2 - Exemplo de declaração de uma Series a partir de uma lista
Fonte: Elaborado pelo autor (2024)

2.2 DataFrame

O DataFrame é uma estrutura de dados bidimensional, semelhante à estrutura de uma tabela em um banco relacional ou uma planilha do Excel. Cada uma das colunas que compõem um DataFrame é uma Series, alinhada por um índice comum.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
dados = {
'Produto': ['A', 'B', 'C'],
'Preço': [100, 200, 300],
'Quantidade': [10, 20, 30]
}

# Criando um DataFrame a partir de um dicionário
df = pd.DataFrame(dados)
```

Código-fonte 3 - Exemplo de declaração de um DataFrame a partir de um dicionário
Fonte: Elaborado pelo autor (2024)

Na criação de DataFrames por meio de dicionários, as chaves assumem a identificação das colunas. O acesso aos dados pode ser realizado pelo nome das colunas ou pelo índice da linha, utilizando a posição ou o rótulo. Além disso, ambos podem ser combinados para chegar a um valor individual na tabela.

```
# Acesso à coluna Produto
coluna = df['Produto']

# Acesso à múltiplas colunas
colunas = df[['Produto', 'Preço']]

# Acesso à primeira linha pelo rótulo
linha = df.loc[0]

# Acesso à primeira linha pela posição
linha = df.iloc[0]

# Acesso a um intervalo de linhas pela posição
df.iloc[0:2]

# Acesso a um único valor pela coluna e linha
a = df['Produto'][0]
print(a) # Saída: 'A'

# Acesso a um único valor pela linha e coluna
b = df.iloc[0]['Produto']
print(b) # Saída: 'A'
```

Código-fonte 4 - Acesso às Series e valores do DataFrame
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 OPERAÇÕES BÁSICAS

A biblioteca Pandas permite realizar uma vasta gama de operações para a manipulação e análise de dados. Essas operações incluem: a filtragem, que possibilita a seleção de subconjuntos com base em condições específicas; a agregação, permitindo resumir dados por meio de funções estatísticas; e a ordenação, que viabiliza a organização de informações em ordem crescente ou decrescente, facilitando a realização de análises comparativas. Além disso, o Pandas oferece recursos avançados para o tratamento de informações ausentes, a combinação de diferentes conjuntos de dados e a transformação de dados através de operações de mapeamento e aplicação de funções personalizadas. Essas operações são fundamentais para a análise exploratória, permitindo aos usuários compreender os padrões, identificar as tendências e extrair *insights* valiosos para o contexto da aplicação. Pandas também é altamente eficiente em operações vetorizadas, permitindo a manipulação de grandes conjuntos de dados em memória de forma rápida, eficaz e precisa.

3.1 Filtragem de dados

A filtragem de dados permite a seleção elementos de um DataFrame ou Series que atendem a certas condições. Essa operação é crucial para isolar amostras ou subconjuntos de dados que são relevantes para análises específicas.

```
# Filtragem de dados com base em uma condição
produtos_caros = df[df['Preço'] > 150]

# Filtragem com múltiplas condições
produtos_caros_quantidade_alta = df[(df['Preço'] > 150) &
                                    (df['Quantidade'] > 20)]
```

Código-fonte 5 - Filtragem de dados
Fonte: Elaborado pelo autor (2024)

3.2 Agregação de dados

Agregação de dados refere-se à aplicação de funções de resumo aos dados, como soma, média, contagem, entre outras. Esse processo é essencial para obter uma visão geral dos dados e identificar padrões ou tendências.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Agregação de dados - soma total
total_vendas = df['Preço'].sum()
print(total_vendas) # Saída: 600

# Agregação de dados - média
media_precos = df['Preço'].mean()
print(media_precos) # Saída: 200

# Agregação de dados por grupo – gera uma tabela sumarizada
vendas_por_produto = df.groupby('Produto').sum()
```

Código-fonte 6 - Agregação de dados
Fonte: Elaborado pelo autor (2024)

3.3 Ordenação dos dados

A ordenação é o processo de organizar os dados em uma sequência específica, seja ela crescente ou decrescente, com base em uma ou mais informações. Esse processo é essencial para várias etapas da análise de dados, pois facilita a comparação entre valores próximos, permitindo a detecção de tendências e anomalias. Ordenar dados pode revelar padrões ocultos e *insights* relevantes que não seriam facilmente visíveis em dados desorganizados.

```
# Ordenação de dados em ordem crescente
df_ordenado = df.sort_values(by='Preço')

# Ordenação de dados em ordem decrescente
df_ordenado_desc = df.sort_values(by='Preço',ascending=False)

# Ordenação de dados por múltiplas colunas
df_ordenado_mult = df.sort_values(by=['Preço', 'Quantidade'])
```

Código-fonte 7 - Ordenação de dados
Fonte: Elaborado pelo autor (2024)

Índice	Produto	Preço	Quantidade
0	A	100	10
1	B	200	20
2	C	300	30

Tabela 1 - DataFrame com ordenação ascendente pelo Produto
Fonte: Elaborado pelo autor (2024)

Índice	Produto	Preço	Quantidade
2	C	300	30
1	B	200	20
0	A	100	10

Tabela 2 - DataFrame com ordenação descendente pelo Produto
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 COMBINAÇÃO DE DATAFRAMES

Combinar DataFrames é uma técnica essencial para integrar informações oriundas de bases distintas. Em Pandas, as principais operações para a combinação de DataFrames incluem a junção (*join*), mesclagem (*merge*) e concatenação (*concat*). Cada uma dessas técnicas é útil em diferentes cenários, como combinar resultados de vários estudos ou construir conjuntos de amostras mais abrangentes a partir de múltiplas fontes. Essas operações são fundamentais para a preparação dos dados, proporcionando uma base sólida para as atividades subsequentes.

4.1 Junção (*Join*)

A junção é utilizada para combinar DataFrames com base em uma chave comum, semelhante a *joins* em bancos de dados relacionais. Isso pode ser feito de várias maneiras, como junção interna, externa, à esquerda ou à direita, determinando como os dados são alinhados e quais linhas são mantidas ou descartadas.

```
# DataFrame de produtos com 'ProdutoID' como índice
df_produtos = pd.DataFrame({
    'ProdutoID': [1, 2, 3, 4],
    'NomeProduto': ['Prod A', 'Prod B', 'Prod C', 'Prod D']
}).set_index('ProdutoID')

# DataFrame de vendas com 'ProdutoID' como índice
df_vendas = pd.DataFrame({
    'VendaID': [101, 102, 103, 104],
    'ProdutoID': [1, 2, 2, 4],
    'Quantidade': [10, 5, 15, 7]
}).set_index('ProdutoID')

# Realizar o join com base no índice 'ProdutoID'
df_combinado = df_vendas.join(df_produtos, how='inner')
```

Código-fonte 8 - Junção de DataFrames
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2 Mesclagem (*Merge*)

A mesclagem oferece um controle mais granular e flexível sobre a combinação de DataFrames, permitindo especificar múltiplas chaves e tipos de junções.

```
# DataFrame de produtos
df_produtos = pd.DataFrame({
    'ProdutoID': [1, 2, 3, 4],
    'NomeProduto': ['Prod A', 'Prod B', 'Prod C', 'Prod D']
})

# DataFrame de vendas
df_vendas = pd.DataFrame({
    'VendaID': [101, 102, 103, 104],
    'ProdutoID': [1, 2, 2, 4],
    'Quantidade': [10, 5, 15, 7]
})

# Realizar o merge com base na coluna 'ProdutoID'
df_combinado = pd.merge(df_vendas, df_produtos,
on='ProdutoID', how='inner')
```

Código-fonte 9 - Mesclagem de DataFrames
Fonte: Elaborado pelo autor (2024)

4.3 Concatenação (*Concat*)

A concatenação é utilizada para empilhar no sentido vertical ou horizontal, geralmente para adicionar novas linhas ou colunas em um DataFrame existente.

```
     # DataFrame de vendas no primeiro semestre
     df_vendas1 = pd.DataFrame({
         'Mês': ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun'],
         'Vendas': [2000, 2200, 2500, 2700, 2600, 2900]
     })

     # DataFrame de vendas no segundo semestre
     df_vendas2 = pd.DataFrame({
         'Mês': ['Jul', 'Ago', 'Set', 'Out', 'Nov', 'Dez'],
         'Vendas': [3100, 3300, 3200, 3400, 3500, 3600]
     })

     # Concatenar DataFrames verticalmente
     df_vendas_completo = pd.concat([df_vendas1, df_vendas2],
ignore_index=True)
```

Código-fonte 10 - Concatenação de DataFrames
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 LEITURA E ESCRITA DE ARQUIVOS

A capacidade de ler e escrever arquivos é fundamental para a manipulação de dados. Pandas suporta diversos formatos de arquivos, como CSV, Excel, SQL e JSON, permitindo uma fácil integração com outras ferramentas e sistemas de armazenamento de dados. Além disso, ele possibilita a persistência dos dados processados, facilitando a continuidade de trabalhos e a reprodução de experimentos.

```
# Escrevendo um DataFrame para um arquivo CSV
df.to_csv('dados.csv', sep=';', index=False)

# Lendo o arquivo CSV criado, para um novo DataFrame
df2 = pd.read_csv('dados.csv', sep=';')
```

Código-fonte 11 - Escrita e leitura de um arquivo CSV
Fonte: Elaborado pelo autor (2024)

Figura 1 - DataFrame armazenado em arquivo CSV com separador utilizando ponto e vírgula
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 APLICAÇÕES PRÁTICAS

Pandas oferece uma vasta gama de funções que tornam a análise de dados mais eficiente e robusta, independentemente da área de aplicação. Suas capacidades incluem a limpeza, a manipulação e a transformação de dados, que são responsáveis pela remoção de inconsistências, formatação de valores, adequação de modelagem, normalização de escalas, agrupamento de amostras etc.

Com uma documentação abrangente e uma comunidade ativa ao redor do mundo, Pandas segue evoluindo constantemente, oferecendo recursos valiosos para a realização de análises de forma intuitiva e confiável, dando suporte às descobertas científicas e a elaboração de estratégias de negócios pautadas em evidências.

6.1 Limpeza e preparação dos dados

A limpeza e a preparação das amostras são etapas cruciais em qualquer pipeline de análise de dados. A biblioteca Pandas oferece ferramentas robustas para tratar informações faltantes, remover duplicatas, normalizar valores e realizar outras operações de pré-processamento necessárias para a análise correta e eficiente das informações. Além disso, Pandas facilita o encadeamento de operações, permitindo realizar múltiplas transformações em uma única linha de código, o que otimiza a execução do pré-processamento. A combinação dessas funcionalidades garante que os dados sejam processados de maneira eficiente e robusta, proporcionando uma base sólida para análises precisas e *insights* confiáveis.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Arrumação interna dos dados

```
data = {
    'Nome': ['Ana', 'Bruno', 'Carlos', 'Daniela', 'Eduardo'],
    'Idade': [23, 35, 45, None, 28],
    'Salário': [50000, 60000, None, 70000, 80000]
}

#Criação de DataFrame com valores nulos (None)
df = pd.DataFrame(data)

# Removendo linhas com valores ausentes
df_cleaned = df.dropna()

# Preenchendo valores ausentes com a média
df['Idade'] = df['Idade'].fillna(df['Idade'].mean())
df['Salário'] = df['Salário'].fillna(df['Salário'].mean())
```

Código-fonte 12 - Preparação e limpeza de dados
Fonte: Elaborado pelo autor (2024)

Nome	Idade	Salário
Ana	23.0	50000.0
Bruno	35.0	60000.0
Eduardo	28.0	80000.0

Tabela 3 - DataFrame com valores nulos excluídos
Fonte: Elaborado pelo autor (2024)

Nome	Idade	Salário
Ana	23.0	50000.0
Bruno	35.0	60000.0
Carlos	45.0	65000.0
Daniela	32.75	70000.0
Eduardo	28.0	80000.0

Tabela 4 - DataFrame com valores nulos preenchidos com a média da coluna
Fonte: Elaborado pelo autor (2024)

6.2 Análise de dados

A biblioteca Pandas é amplamente utilizada para realizar análises de dados devido à sua capacidade de processar grandes volumes de informações de forma eficiente. Essa biblioteca executa de forma simples os cálculos de estatísticas descritivas e as operações de agregação, essenciais para entender o contexto analisado (McKinney, 2018).

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import numpy as np

# Simulação de dados de vendas
data = {
    'Dia': range(1, 31),
    'Vendas': np.random.randint(100, 200, size=30)
}

df_vendas = pd.DataFrame(data)

# Estatísticas descritivas
print(df_vendas.describe())
```

Código-fonte 13 - Análise estatística descritiva
Fonte: Elaborado pelo autor (2024)

	Dia	Vendas
Count	30.0	30.0
Mean	15.5	143.9
Std	8.803408	28.243217
Min	1.0	101.0
25%	8.25	124.75
50%	15.5	142.5
75%	22.75	162.25
Max	30.0	199.0

Tabela 5 - Resultado obtido com execução do método *describe*
Fonte: Elaborado pelo autor (2024)

Dica: Utilize os métodos *head* e *tail* antes de iniciar a análise para se familiarizar com os dados que estão sendo manipulados no contexto da aplicação.

Os métodos *head* e *tail* permitem verificar as extremidades do conjunto de dados, exibindo os primeiros e os últimos registros, respectivamente, visando facilitar a identificação dos valores e formatos presentes na amostra analisada. Por padrão, esses métodos retornam apenas cinco registros, porém, esse montante pode ser alterado informando o parâmetro *n* com a quantidade desejada.

6.3 Análise de séries temporais

A análise de séries temporais é uma área da Estatística e da Ciência de Dados que se concentra em estudar dados coletados ou observados sequencialmente ao longo do tempo. O Pandas fornece ferramentas úteis para manipulação e análise de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

dados temporais, facilitando tarefas como *resampling*, cálculo de médias móveis, análise de sazonalidade e tendências presentes nas amostras. Séries temporais são especialmente importantes em campos como finanças, economia, meteorologia e muitas outras áreas no qual o tempo é uma variável crucial. Além disso, esse método de análise geralmente antecede a implementação de modelos mais robustos baseados em algoritmos de Machine Learning (Kaggle, 2024).

```
     import numpy as np

     # Criando uma série temporal de exemplo
     datas  =  pd.date_range(start='2023-01-01',  periods=100,
freq='D')

     # Série temporal acumulada com ruído
     dados = np.random.randn(100).cumsum()
     ts = pd.Series(dados, index=datas)

     # Calculando a média móvel de 3 dias
     rolling_mean_3 = ts.rolling(window=3).mean()

     # Exibindo os top 10 resultados da média móvel de 3 dias
     print(rolling_mean_3.head(10))
```

Código-fonte 14 - Análise de séries temporais com média móvel
Fonte: Elaborado pelo autor (2024)

Datas	Média
2023-01-01	NaN
2023-01-02	NaN
2023-01-03	0.795866
2023-01-04	1.008832
2023-01-05	0.393313
2023-01-06	-0.180192
2023-01-07	-0.342459
2023-01-08	0.203978
2023-01-09	0.617952
2023-01-10	0.577554

Tabela 6 - Top 10 registros com média móvel de três dias
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.4 Integração nativa com visualização de dados

Pandas inclui funcionalidades integradas para a geração de gráficos, permitindo a visualização direta dos dados armazenados em DataFrames e Series. Utilizando uma interface simplificada, baseada em Matplotlib, os usuários podem rapidamente criar gráficos de linha, barras, histogramas, *scatter plots* e muitos outros tipos de visualizações com chamadas de métodos simples como *plot()*. Essas funções facilitam a análise exploratória, fornecendo *insights* visuais rápidos sobre distribuições, tendências e correlações dentro dos dados. Além disso, a biblioteca permite a personalização dos gráficos por meio de parâmetros que ajustam aspectos como títulos, rótulos de eixos, cores e estilos de linha. Com a capacidade de gerar gráficos a partir de operações de agrupamento e agregação, a biblioteca Pandas oferece uma maneira eficaz de resumir e comunicar informações complexas de maneira visual. Esta integração de visualização de dados diretamente na biblioteca de análise de dados permite um fluxo de trabalho contínuo, onde a manipulação e a visualização de dados são realizadas de forma eficiente e coesa.

```
# Dados de exemplo de vendas mensais
data = {
    'Mês': ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun'],
    'Vendas': [2500, 2700, 3000, 3200, 3100, 3300]
}

# Criar DataFrame e plotar um gráfico de linha
df = pd.DataFrame(data)
df.plot(x='Mês', y='Vendas', marker='o', color='b')
```

Código-fonte 15 - Script para criação de gráfico diretamente a partir do DataFrame
Fonte: Elaborado pelo autor (2024)

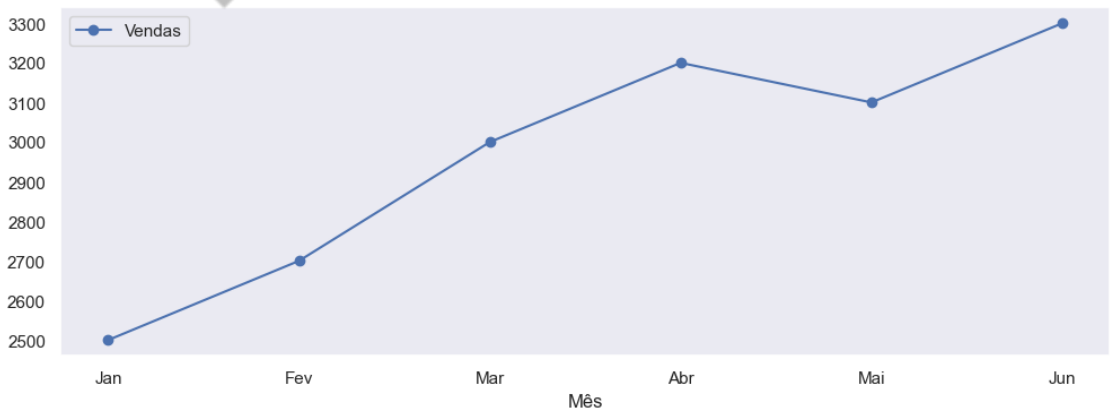


Figura 2 - Gráfico gerado diretamente a partir do DataFrame Pandas
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Pandas é uma biblioteca essencial para a manipulação e análise de dados em Python, destacando-se por sua eficiência, flexibilidade e abrangência. Com suas poderosas ferramentas para limpeza, transformação, filtragem, agregação e ordenação de dados, Pandas consegue simplificar processos complexos, tornando a análise mais acessível e intuitiva. Sua capacidade de lidar com grandes volumes de informações e integrá-los com outras bibliotecas e formatos, o torna indispensável em diversos contextos, desde a ciência de dados até aplicações empresariais.

A comunidade ativa e a documentação extensa garantem que os usuários possam aproveitar ao máximo suas funcionalidades, aprimorando continuamente suas habilidades de análise de dados. Em resumo, Pandas não apenas facilita a exploração e compreensão de dados, mas também habilita a tomada de decisões baseadas em informações precisas e *insights* profundos, solidificando seu papel como uma ferramenta fundamental na análise de dados moderna.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

COMPLETE Guide on Time Series Analysis in Python. **Kaggle**, 30 ago. 2020. Disponível em: <https://www.kaggle.com/>. Acesso em: 14 ago. 2024.

MCKINNEY, W. **Python para análise de dados**. Novatec Editora, 2018.

PANDAS documentation. **PyData.org**, 10 de abril de 2024. Disponível em: <https://pandas.pydata.org/>. Acesso em: 14 ago. 2024.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com